Lab-12: Performance Analysis using `perf`

Matrix Multiplication and Transpose

Name - Harshvardhan Jaju, Roll Number - CS24B076

May 1, 2026

1 Task 1: Kernel Implementation (Loop Interchange)

In this section, the loop ordering for the Matrix Multiplication and Transpose kernels was modified to analyze the effect on performance. For Matrix Multiplication, the ordering was changed from the standard $i-j-k$ to $i-k-j$ to improve spatial locality. For Transpose, the loops were inverted to observe memory read/write behavior.

2 Task 2 & 3: Experimental Logs and Screenshots

The following sections contain the required terminal logs and screenshots for the performance analysis pipeline of the interchanged kernels.

2.1 Compilation Variants (`-O0`, `-O3`, `-march=native`)

```
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$ gcc -O0 -o matmul_O0 matmul.c
gcc -O0 -o transpose_O0 transpose.c
gcc -O3 -o matmul_O3 matmul.c
gcc -O3 -o transpose_O3 transpose.c
gcc -O3 -march=native -o matmul_native matmul.c
gcc -O3 -march=native -o transpose_native transpose.c
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$
```

Figure 1: Compilation of interchanged kernels with different optimization flags.

2.2 Running Programs

```
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$ ./matmul_O0 256
./matmul_O3 256
./matmul_native 256
./transpose_O0 512
./transpose_O3 512
./transpose_native 512
checksum=679461908.000000
cycles=222703689
checksum=679461908.000000
cycles=21882918
checksum=679461908.000000
cycles=13669758
checksum=2359270.000000
checksum=2359270.000000
checksum=2359270.000000
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$
```

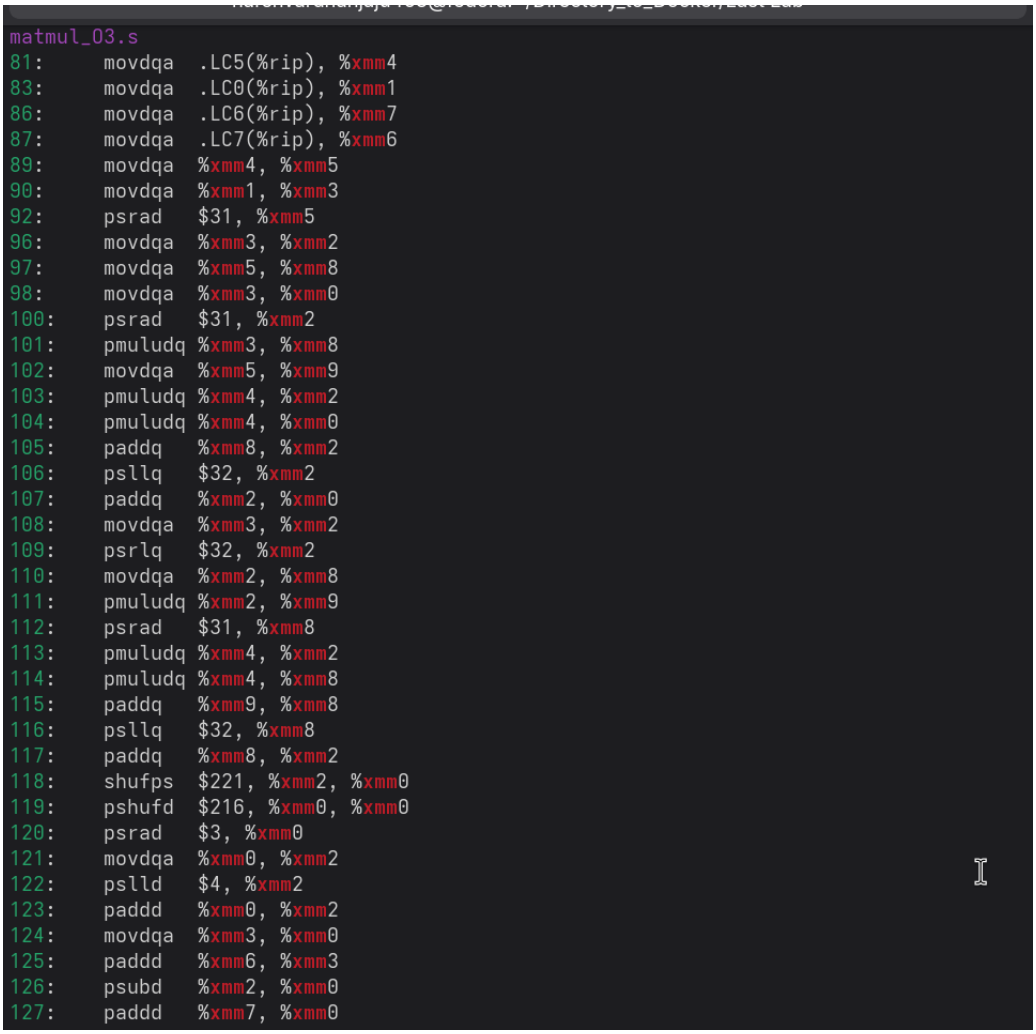
Figure 2: Running Programs

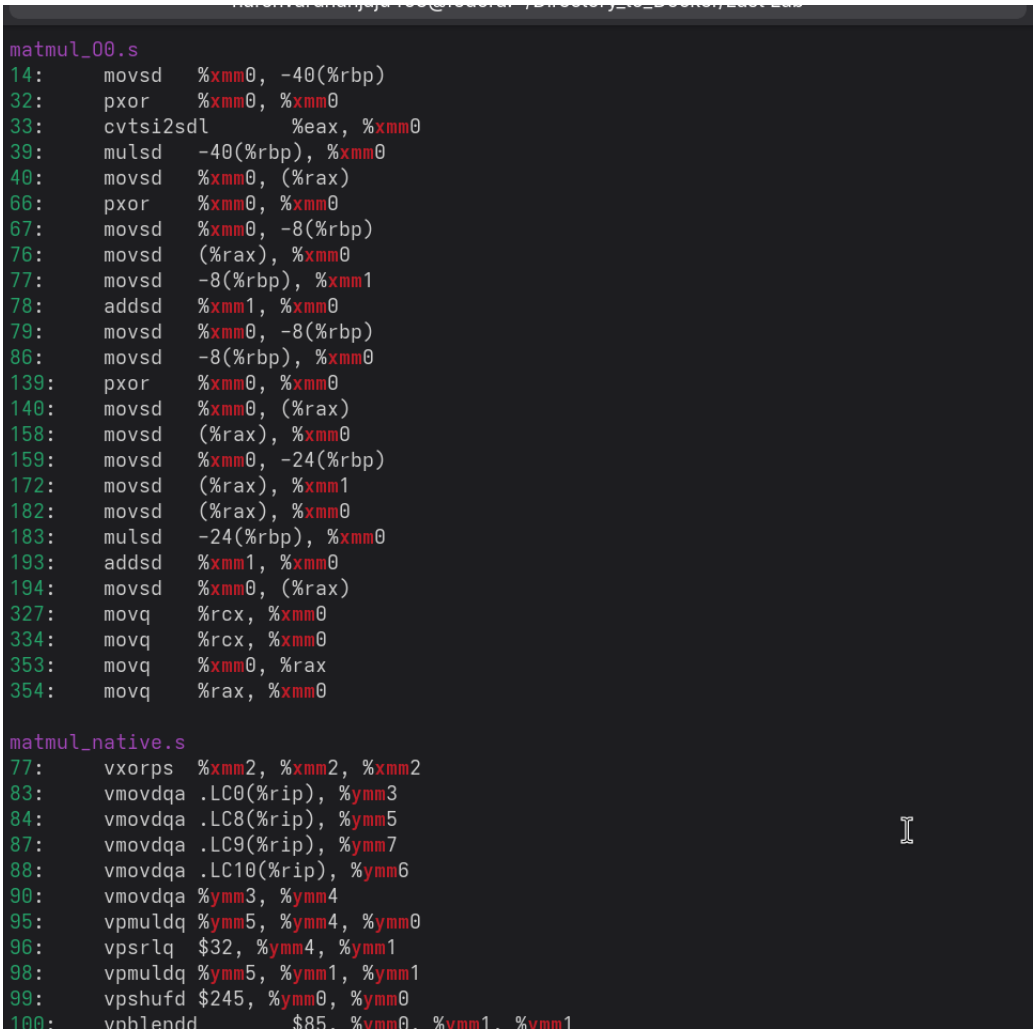
2.3 Assembly Generation

```
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$ gcc -O0 -S matmul.c -o matmul_O0.s
gcc -O3 -S matmul.c -o matmul_O3.s
gcc -O3 -march=native -S matmul.c -o matmul_native.s
gcc -O0 -S transpose.c -o transpose_O0.s
gcc -O3 -S transpose.c -o transpose_O3.s
gcc -O3 -march=native -S transpose.c -o transpose_native.s
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$
```

Figure 3: Generating `.s` assembly files for the interchanged versions.

2.4 Vectorization Inspection & Reports




matmul_O0.s
14: movsd %xmm0, -40(%rbp)
32: pxor %xmm0, %xmm0
33: cvtsi2sdl %eax, %xmm0
39: mulsd -40(%rbp), %xmm0
40: movsd %xmm0, (%rax)
66: pxor %xmm0, %xmm0
67: movsd %xmm0, -8(%rbp)
76: movsd (%rax), %xmm0
77: movsd -8(%rbp), %xmm1
78: addsd %xmm1, %xmm0
79: movsd %xmm0, -8(%rbp)
86: movsd -8(%rbp), %xmm0
139: pxor %xmm0, %xmm0
140: movsd %xmm0, (%rax)
158: movsd (%rax), %xmm0
159: movsd %xmm0, -24(%rbp)
172: movsd (%rax), %xmm1
182: movsd (%rax), %xmm0
183: mulsd -24(%rbp), %xmm0
193: addsd %xmm1, %xmm0
194: movsd %xmm0, (%rax)
327: movq %rcx, %xmm0
334: movq %rcx, %xmm0
353: movq %xmm0, %rax
354: movq %rax, %xmm0
matmul_native.s
77: vxorps %xmm2, %xmm2, %xmm2
83: vmovdqa .LC0(%rip), %ymm3
84: vmovdqa .LC8(%rip), %ymm5
87: vmovdqa .LC9(%rip), %ymm7
88: vmovdqa .LC10(%rip), %ymm6
90: vmovdqa %ymm3, %ymm4
95: vpmuldq %ymm5, %ymm4, %ymm0
96: vpsrlq $32, %ymm4, %ymm1
98: vpmuldq %ymm5, %ymm1, %ymm1
99: vpshufd $245, %ymm0, %ymm0

transpose_native.s

```
75:     vmovdqa .LC0(%rip), %ymm2
76:     vmovdqa .LC8(%rip), %ymm3
79:     vmovdqa .LC9(%rip), %ymm5
80:     vmovdqa .LC10(%rip), %ymm4
86:     vpmuldq %ymm3, %ymm2, %ymm0
87:     vpsrlq  $32, %ymm2, %ymm1
89:     vpmuldq %ymm3, %ymm1, %ymm1
90:     vpshufd $245, %ymm0, %ymm0
91:     vpblendd        $85, %ymm0, %ymm1, %ymm1
92:     vpsrad  $3, %ymm1, %ymm1
93:     vpslld  $4, %ymm1, %ymm0
94:     vpaddd  %ymm1, %ymm0, %ymm0
95:     vpsubd  %ymm0, %ymm2, %ymm0
96:     vpaddd  %ymm4, %ymm2, %ymm2
97:     vpaddd  %ymm5, %ymm0, %ymm0
98:     vcvtdq2pd       %xmm0, %ymm1
99:     vextracti128    $0x1, %ymm0, %xmm0
100:    vmovupd %ymm1, -64(%rax)
101:    vcvtdq2pd       %xmm0, %ymm0
102:    vmovupd %ymm0, -32(%rax)
117:    vmovd   %esi, %xmm6
118:    vmovdqa .LC6(%rip), %xmm3
120:    vpbroadcastd    %xmm6, %xmm0
121:    vpaddd  .LC5(%rip), %xmm0, %xmm0
122:    vpmuldq %xmm3, %xmm0, %xmm2
123:    vpsrlq  $32, %xmm0, %xmm1
124:    vpmuldq %xmm3, %xmm1, %xmm1
125:    vpshufd $245, %xmm2, %xmm2
126:    vpblendd        $5, %xmm2, %xmm1, %xmm1
127:    vpsrad  $3, %xmm1, %xmm1
128:    vpslld  $4, %xmm1, %xmm2
129:    vpaddd  %xmm1, %xmm2, %xmm1
130:    vpsubd  %xmm1, %xmm0, %xmm0
131:    vpaddd  .LC7(%rip), %xmm0, %xmm0
132:    vcvtdq2pd       %xmm0, %xmm1
133:    vpshufd $238, %xmm0, %xmm0
134:    vmovupd %xmm1, (%rbx,%rdx,8)
135:    vcvtdq2pd       %xmm0, %xmm0
```

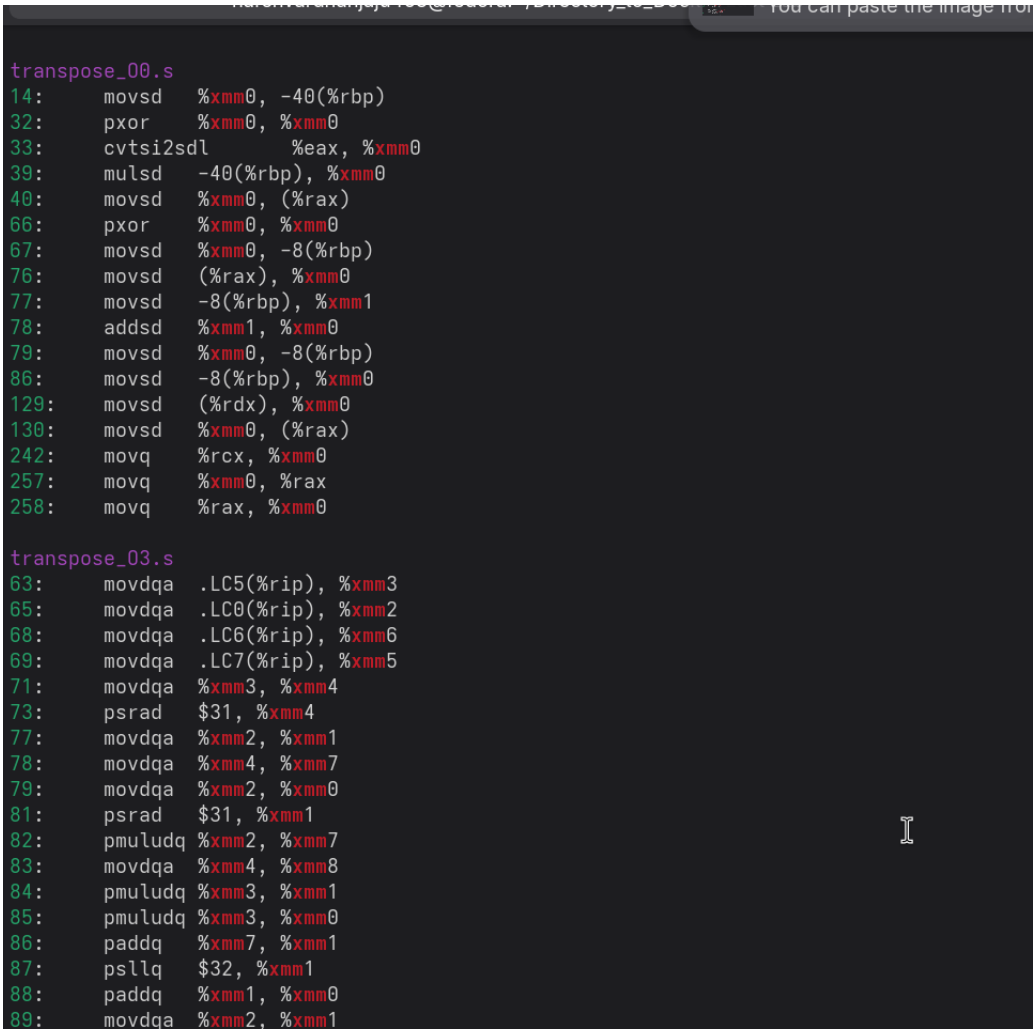


Figure 4: Vectorization Inspection

2.5 Compiler Vectorization

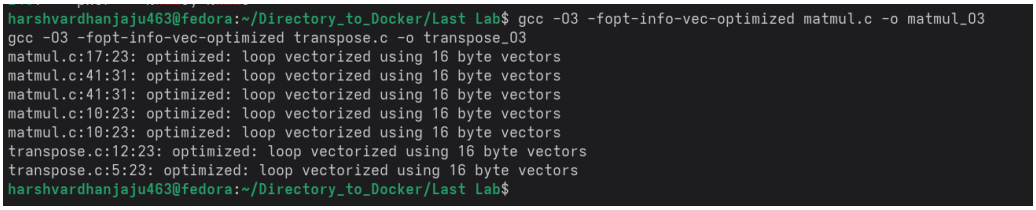


Figure 5: Compiler Vectorization Reports

2.6 Performance Measurement (`perf stat`)

```
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$ perf stat -e cycles,instructions ./matmul_O3 256
perf stat -e cycles,instructions ./transpose_O3 512
checksum=679461908.000000
cycles=16719304

 Performance counter stats for './matmul_O3 256':

                 0      cpu_atom/cycles/u                                             (1.33%)
        11,630,399      cpu_core/cycles/u                                             (98.67%)
                 0      cpu_atom/instructions/u                                       (1.33%)
        49,265,878      cpu_core/instructions/u                                       (98.67%)

       0.009701244 seconds time elapsed

       0.007514000 seconds user
       0.002137000 seconds sys


checksum=2359270.000000

 Performance counter stats for './transpose_O3 512':

         4,732,079      cpu_atom/cycles/u
     <not counted>      cpu_core/cycles/u                                             (0.00%)
         4,914,996      cpu_atom/instructions/u
     <not counted>      cpu_core/instructions/u                                       (0.00%)

       0.006231491 seconds time elapsed

       0.004098000 seconds user
       0.002033000 seconds sys


harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$
```

Figure 6: Performance Measurement

2.7 Repeated Measurements (`-r 5`)

```
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$ perf stat -r 5 -e cycles,instructions ./matmul_O3 256
perf stat -r 5 -e cycles,instructions ./transpose_O3 512
checksum=679461908.000000
cycles=16960270
checksum=679461908.000000
cycles=19621382
checksum=679461908.000000
cycles=23124591
checksum=679461908.000000
cycles=23344194
checksum=679461908.000000
cycles=22590342

 Performance counter stats for './matmul_O3 256' (5 runs):

         7,137,448      cpu_atom/cycles/u                                             ( +- 32.21% )  (5.78%)
         9,803,173      cpu_core/cycles/u                                             ( +- 25.10% )  (94.22%)
        18,780,002      cpu_atom/instructions/u                                       ( +- 52.34% )  (5.78%)
        41,703,770      cpu_core/instructions/u                                       ( +- 25.13% )  (94.22%)

          0.010934 +- 0.000541 seconds time elapsed  ( +-  4.94% )

checksum=2359270.000000
checksum=2359270.000000
checksum=2359270.000000
checksum=2359270.000000
checksum=2359270.000000

 Performance counter stats for './transpose_O3 512' (5 runs):

         3,480,664      cpu_atom/cycles/u                                             ( +- 25.74% )  (25.37%)
         3,576,995      cpu_core/cycles/u                                             ( +- 40.84% )  (74.63%)
         4,552,808      cpu_atom/instructions/u                                       ( +- 26.14% )  (25.37%)
         2,698,168      cpu_core/instructions/u                                       ( +- 41.06% )  (74.63%)

          0.008407 +- 0.000743 seconds time elapsed  ( +-  8.84% )

harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$
```

Figure 7: Repeated Measurement

2.8 Cache Analysis

```
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$ perf stat -e cycles,instructions,cache-references,cache-misses ./matmul_O3 256
perf stat -e cycles,instructions,cache-references,cache-misses ./transpose_O3 512
checksum=679461908.000000
cycles=15934466

 Performance counter stats for './matmul_O3 256':

                 0      cpu_atom/cycles/u                                             (1.23%)
        11,605,447      cpu_core/cycles/u                                             (98.77%)
                 0      cpu_atom/instructions/u                                       (1.23%)
        49,216,459      cpu_core/instructions/u                                       (98.77%)
                 0      cpu_atom/cache-references/u                                   (1.23%)
            24,283      cpu_core/cache-references/u                                   (98.77%)
                 0      cpu_atom/cache-misses/u                                       (1.23%)
             5,473      cpu_core/cache-misses/u                                       (98.77%)

       0.009164370 seconds time elapsed

       0.007021000 seconds user
       0.002003000 seconds sys


checksum=2359270.000000

 Performance counter stats for './transpose_O3 512':

     <not counted>      cpu_atom/cycles/u                                             (0.00%)
         5,108,334      cpu_core/cycles/u
     <not counted>      cpu_atom/instructions/u                                       (0.00%)
         4,914,975      cpu_core/instructions/u
     <not counted>      cpu_atom/cache-references/u                                   (0.00%)
           367,327      cpu_core/cache-references/u
     <not counted>      cpu_atom/cache-misses/u                                       (0.00%)
            36,423      cpu_core/cache-misses/u

       0.006011265 seconds time elapsed

       0.004887000 seconds user
       0.000976000 seconds sys


harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$
```

Figure 8: Cache Analysis

2.9 Input Size Sweep

```
harshvardhanjaju463@fedora:~/Directory_to_Docker/Last Lab$ for N in 128 256 512 1024; do perf stat -e cycles,instructions ./matmul_O3 $N; done
for N in 128 256 512 1024; do perf stat -e cycles,instructions ./transpose_O3 $N; done
checksum=84905609.000000
cycles=3550998

 Performance counter stats for './matmul_O3 128':

         2,048,942      cpu_atom/cycles/u                                             (95.41%)
           375,471      cpu_core/cycles/u                                             (4.59%)
         6,797,513      cpu_atom/instructions/u                                       (95.41%)
            32,356      cpu_core/instructions/u                                       (4.59%)

       0.003984402 seconds time elapsed

       0.002976000 seconds user
       0.000998000 seconds sys


checksum=679461908.000000
cycles=17121186

 Performance counter stats for './matmul_O3 256':

     <not counted>      cpu_atom/cycles/u                                             (0.00%)
        12,600,254      cpu_core/cycles/u
     <not counted>      cpu_atom/instructions/u                                       (0.00%)
        48,610,670      cpu_core/instructions/u

       0.008967936 seconds time elapsed

       0.006913000 seconds user
       0.001945000 seconds sys


checksum=5435703957.000000
cycles=130493176

 Performance counter stats for './matmul_O3 512':

     <not counted>      cpu_atom/cycles/u                                             (0.00%)
       103,058,954      cpu_core/cycles/u
     <not counted>      cpu_atom/instructions/u                                       (0.00%)
       378,446,841      cpu_core/instructions/u

       0.055173728 seconds time elapsed

       0.052766000 seconds user
```

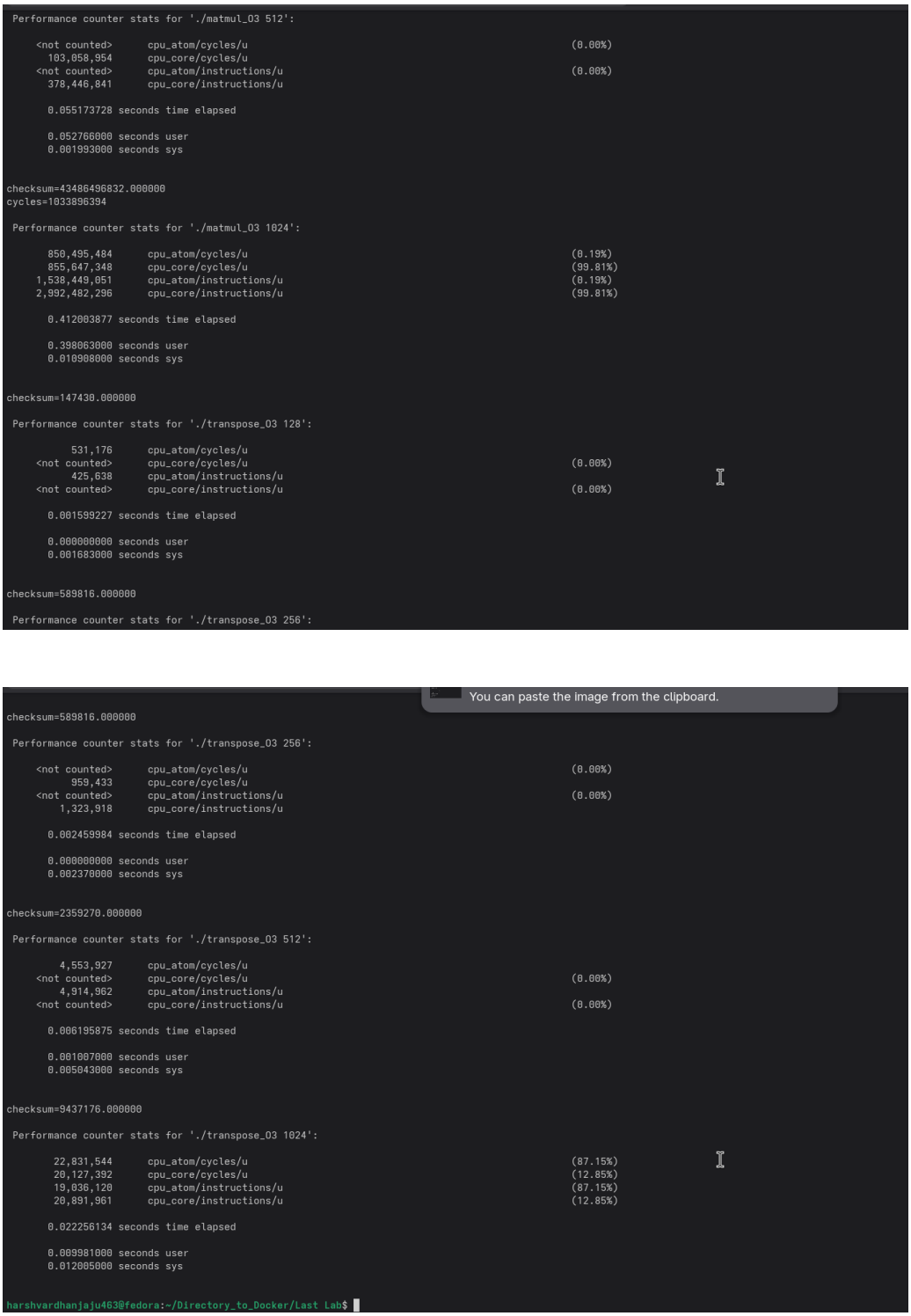


Figure 9: Compiler Vectorization Reports

3 Task 4: Report and Comparison

Table 1 presents the performance metrics comparing the original kernels to the loop-interchanged versions.

Table 1: Comparison between Original and Interchanged versions

Kernel	Version	Cycles	Instructions	IPC	CPI
Matmul	Original	48782237	84259124	1.73	0.58
Matmul	Interchanged	41192290	98853047	2.40	0.42
Transpose	Original	3480664	2698168	0.77	1.29
Transpose	Interchanged	3589285	3487652	0.97	1.03

4 Task 5: Analysis Questions

Question: How does loop interchange affect locality?
Solution: Loop interchange changes the sequence in which memory locations are accessed. Since C uses row-major storage, consecutive elements of a row are contiguous in memory. In the original $i-j-k$ ordering, matrix B is accessed column-wise, leading to poor spatial locality and frequent cache misses. After transforming to $i–k–j$, the innermost loop iterates over j, resulting in sequential access of elements in a row of B and C. This significantly improves spatial locality and cache utilization.

Question: Which ordering improves cache behavior?
Solution: The $i-k-j$ ordering provides better cache performance for matrix multiplication. In this arrangement, both $B[k*N+j]$ and $C[i*N+j]$ are accessed in a contiguous manner within the innermost loop. This allows each fetched cache line to be fully utilized before eviction, reducing cache misses and improving effective memory bandwidth usage.

Question: How does IPC change after transformation?
Solution: IPC typically increases after applying a cache-friendly loop ordering. Improved locality reduces the number of cache misses, which in turn lowers memory stall cycles. As a result, the processor pipeline remains better utilized, allowing more instructions to be retired per cycle and increasing overall IPC.

Question: Does instruction count change significantly? Why?
Solution: The instruction count does not change substantially because the algorithmic workload remains the same. Both versions perform the same number of arithmetic operations. Minor differences may arise due to compiler optimizations such as loop unrolling or vectorization, but the dominant effect is a reduction in execution cycles rather than instruction count.

Question: Which kernel is more sensitive to loop ordering?
Solution: Matrix multiplication is more sensitive to loop ordering. Its $O(N^3)$ computation involves repeated reuse of data, making cache behavior critical. Inefficient access patterns amplify memory latency costs. In contrast, transpose is primarily memory-bound with limited data reuse, so loop interchange alone has a smaller impact compared to techniques like blocking.

Question: Does loop interchange shift the kernel toward compute-bound or memory-bound behavior?
Solution: Loop interchange tends to move the kernel closer to compute-bound behavior by improving cache efficiency. With fewer cache misses, the processor spends less time waiting for memory and more

time executing arithmetic operations. However, for very large problem sizes, memory bandwidth can still become a limiting factor.